



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

WEASOLVE:

Weather-Based Food Recommendation System

SDG #3 - Good Health and Well-being

A Final project for Computer Programming II

Presented to

Ms. Justin Louise R. Neypes

Instructor

Submitted by

V of Spades

Paulino, Eurie V.

Alvarez, Jerome Oliver P.

Beli, Jhon Rey B.

Malabanan, Justine Lyle R.

BSIT 1.3

May 17, 2026



TABLE OF CONTENTS

I. INTRODUCTION	4
1.1 Project Overview and Target SDG	4
1.2 Problem Statement	6
II. REQUIREMENTS ANALYSIS	7
2.1 Functional Requirements	7
2.2 Non-Functional Requirements	10
III. DESIGN SPECIFICATION	15
3.1 Algorithmic Flowcharts	16
Figure 1. WEASOLVE Main System Flowchart	17
Figure 2. Smart Ranked Recommendation Engine Flowchart	18
3.2 File Structure Diagram	19
3.3 Database Format	21
3.4 Smart Ranked Recommendation Engine Design	22
IV. TEST CASES AND VALIDATION	24
V. CONCLUSION AND CONTRIBUTIONS	26
5.1 Project Conclusion	26
5.2 Individual Contributions	27



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

5.3 AI Use Disclosure 27

VI. REFERENCES 29



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

I. INTRODUCTION

1.1 Project Overview and Target SDG

WEASOLVE, or the Weather-Based Food Recommendation System, is a C++ console-based application designed to help users choose food items based on their current weather conditions, budget preferences, and available money. The system is developed as a final project for CC103 - Computer Programming 2. It demonstrates the practical application of modular programming, object-oriented programming, file handling, input validation, CRUD operations, and rule-based recommendation logic.

The main purpose of WEASOLVE is to provide a structured, practical food recommendation tool that not only displays stored food records but also evaluates each food item using a scoring model. The system accepts the user's current weather conditions, selected budget level, and available funds, then processes all stored food records using a Smart Ranked Recommendation Engine. This engine applies rule-based scoring using three major criteria: weather match, budget match, and affordability. After scoring each eligible food item, the system displays ranked recommendations with scores and explanations.

The project is aligned with Sustainable Development Goal 3: Good Health and Well-Being. SDG 3 focuses on promoting healthy lives and well-being for all. Although WEASOLVE is not a medical or clinical nutrition system, it supports this goal by helping users make more practical and situationally appropriate food choices. Weather conditions can affect food preferences, comfort, and everyday decision-making. Budget limitations and available money also influence whether a food recommendation is realistic for the user. By combining weather suitability, budget category, and affordability, WEASOLVE promotes practical decision-making that supports user well-being in daily life.

The system uses an external text file database named database.txt, which stores pre-existing food records in a pipe-delimited format. Each record contains a food ID, food name, weather category, budget level, price, restaurant name, and health note. The database contains 50 pre-existing records, exceeding



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

the minimum requirement of 20 records. These records are loaded into a vector<FoodItem> collection when the program starts. Using file handling allows the system to preserve data between program executions, while using a vector allows the program to manage records dynamically at runtime.

WEASOLVE also includes an admin panel protected by a PIN. Through this admin panel, authorized users can add, update, and delete food records. The system also includes functions for viewing all records, searching food records, saving favorite foods, viewing favorite foods, and generating a summary report. These features allow the system to function not only as a recommendation tool but also as a complete record management system.

The most important feature of the project is the Smart Ranked Recommendation Engine. This engine demonstrates polymorphism through the abstract RecommendationRule class and its derived rule classes: WeatherMatchRule, BudgetMatchRule, and AffordabilityRule. The detailed scoring model, threshold, and design rationale are discussed further in Section 3.4.

Only food items with a score of 70 or higher are included in the recommendation results. In addition, the system applies an affordability hard filter, meaning that food items priced above the user's available funds are skipped before scoring. This makes the recommendation results more practical because the system does not recommend food that the user cannot actually afford.

The HealthNote field is included only as descriptive information and is discussed further in the design section to clarify that WEASOLVE is not a medical recommendation system.

Overall, WEASOLVE is designed to satisfy the technical requirements of the final project while also presenting a unique and defensible feature through its Smart Ranked Recommendation Engine. It applies the major lessons from CC103, including decision control, loops, functions, arrays and vectors, structs, object-oriented programming, polymorphism, and file handling.



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

1.2 Problem Statement

Many students and everyday consumers decide what to eat based only on cravings, convenience, or what is immediately available. However, food choices are often influenced by practical factors, such as current weather, personal budget constraints, and available funds. Without a structured decision-making tool, users may spend unnecessary time comparing options or may choose food that does not match their current situation.

A common problem with simple food listing systems is that they only display available records without analyzing whether a food item is actually suitable for the user. For example, a food item may match the weather but may be too expensive. Another food item may be affordable but may not match the user's selected budget category. A system that only lists food records cannot properly prioritize which food item is most relevant.

WEASOLVE addresses this problem by implementing a Smart Ranked Recommendation Engine. Instead of using a single condition, the system evaluates each food item using multiple rule-based criteria. The recommendation engine checks whether the food matches the current weather, the user's selected budget level, and the user's available funds. These factors are converted into a 100-point scoring model, allowing the system to rank food recommendations from strongest to weakest.

Another issue in food recommendation systems is the possibility of recommending food that the user cannot afford. This weakens the recommendation's usefulness because the result may be technically relevant but practically unusable. WEASOLVE solves this through an affordability hard filter. If the item's price exceeds the user's available funds, it is immediately skipped and not included in the final recommendation list.

The system also addresses data management problems. Without CRUD operations and file handling, food records would be difficult to update, maintain, or reuse in future sessions. WEASOLVE solves this by storing food records in an external database.txt file and managing them through a



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

FoodManager class. The admin panel allows authorized users to add, update, and delete food records, while file handling ensures that changes are saved and can be loaded again.

Therefore, WEASOLVE exists to solve three main problems:

1. Users need a practical way to choose food based on weather, budget, and available money.
2. Simple food lists do not rank recommendations or explain why a food item is suggested.
3. Food records need to be stored, updated, searched, and preserved through file handling.

By solving these problems, WEASOLVE provides a complete C++ console application that is not only functional but also analytical, modular, and aligned with the goals of SDG 3.

II. REQUIREMENTS ANALYSIS

Requirements analysis identifies the functional and non-functional requirements the WEASOLVE system must satisfy. Functional requirements specify the services and operations the system performs, while non-functional requirements define the system's quality standards, such as robustness, modularity, readability, and efficiency.

The requirement format used in this document follows a strong evidence-based style, similar to the sample final project documentation, in which each requirement is linked to its system implementation and source files. This makes the documentation easier to verify during project evaluation.

FR ID	Requirement	Rubric Definition	System Implementation and Evidence from Code	Source Files / Functions



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

FR1	Automated Data Initialization	Upon execution, the system must automatically load pre-existing records from an external database file and instantiate them into an internal collection. The system must include at least 20 records.	When the program starts, main.cpp creates a FoodManager manager object and calls chooseDatabasePath() to locate the external database.txt file. The system then calls manager.loadFromFile(databasePath) to read food records from the database. Each valid line is parsed and converted into a FoodItem object. These objects are stored in a vector<FoodItem> collection inside the FoodManager class. The database contains 50 pre-existing food records, exceeding the minimum requirement of 20 records. If the database file fails to load, the program displays a warning message but continues running safely.	main.cpp: main(), chooseDatabasePath(); FoodManager.cpp: loadFromFile(); FoodItem.h, FoodItem.cpp; database.txt
FR2	Record Management and CRUD Operations	The system must implement Create, Read, Update, Delete, and Search operations for its main records.	WEASOLVE includes an admin panel that allows authorized users to manage food records. The addFood() function creates a new food record after collecting the food name, weather category, budget level, price, restaurant, and health note. The viewAllFoods() function displays all stored food records. The searchFood() function allows users to search records by ID, name, or restaurant. The updateFood() function modifies existing food record attributes. The deleteFood() function removes a selected food item from the vector after validation. These operations directly modify the internal vector<FoodItem> collection.	main.cpp: runAdminPanel(), displayAdminMenu(); FoodManager.h; FoodManager.cpp: addFood(), viewAllFoods(), searchFood(), updateFood(), deleteFood()



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

FR3	Smart Ranked Recommendation Engine	The system must implement a value-added computation engine that processes stored data and produces meaningful recommendations or computed results.	The Smart Ranked Recommendation Engine is the core value-added computation feature of WEASOLVE. When the user selects menu option 5, the system calls <code>manager.recommendSmartRanking(currentWeather)</code> . This function collects the user's budget level and available money, creates a <code>UserPreference</code> object, and passes the food records to <code>RecommendationEngine::recommend()</code> . The engine evaluates each food item using polymorphic rule objects: <code>WeatherMatchRule</code> , <code>BudgetMatchRule</code> , and <code>AffordabilityRule</code> . The scoring model awards 50 points for weather match, 30 points for budget match, and 20 points for affordability. Only food items with a score of 70 or higher are stored as <code>RecommendationResult</code> objects.	main.cpp: case 5; FoodManager.cpp: <code>recommendSmartRanking();</code> RecommendationEngine.h, RecommendationEngine.cpp: <code>RecommendationEngine(),</code> <code>recommend();</code> RecommendationRule.h, RecommendationRule.cpp; UserPreference.h; RecommendationResult.h
FR4	Affordability Filtering	The system must prevent impractical recommendations by ensuring that the recommended food items are within the user's budget.	Before the recommendation rules are applied, the engine checks whether the price of the current food item exceeds the user's available money. If <code>food.getPrice() > preference.availableMoney</code> , the system continues to skip that food item. This prevents unaffordable food from being scored or displayed. The affordability rule also contributes 20 points when the food price is within the user's available money. This makes the recommendation practical and defensible.	RecommendationEngine.cpp: <code>recommend();</code> RecommendationRule.cpp: <code>AffordabilityRule::score(),</code> <code>AffordabilityRule::explain()</code>
FR5	Persistent State Storage	The system must synchronize internal memory with an external file to preserve data across program executions.	Food records are stored in <code>database.txt</code> . When the program starts, records are loaded from the file into the internal vector. When the user selects Save and Exit, the program calls <code>manager.saveToFile(databasePath)</code> , which writes the current food records back to the external database file. This ensures that added, updated, and deleted records can be preserved after the program terminates.	main.cpp: case 10; FoodManager.cpp: <code>loadFromFile(), saveToFile();</code> database.txt



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

FR6	Looped Navigational Menu System	The system must provide a looped menu interface that allows users to move between modules without terminating the program.	The main program uses a do-while loop that continues running until the user selects option 10, Save and Exit. The menu contains 10 options: enter current weather, view simple weather recommendations, save favorite food, view favorite foods, use smart ranked recommendation, view all records, search food record, access admin panel, generate summary report, and save and exit. The switch statement routes each validated menu choice to the correct module.	main.cpp: displayMainMenu(), main(), switch(choice)
FR7	Input Validation System	The system must prevent invalid input from causing crashes or infinite loops.	WEASOLVE uses a dedicated InputValidator class to handle safe user input. The system validates integer menu choices, positive double values, weather categories, budget categories, and non-empty text fields. It also supports cancel commands, such as 0 for returning to the previous menu. Input stream recovery techniques, such as clearing invalid input states and flushing the input buffer, are used to prevent crashes caused by type-mismatch errors.	InputValidator.h; InputValidator.cpp: getIntInRange(), getWeatherInputAllowCancel(), getBudgetInputAllowCancel(), getPositiveDoubleAllowCancel(), getNonEmptyLine(), isCancelCommand(), toLowerCase()
FR8	Favorites and Summary Report	The system must include additional user support features that improve usability and reporting.	The system allows users to save selected recommended foods as favorites through saveFavoriteFood(currentWeather) and later view them using viewFavorites(). It also provides a summary report through generateSummaryReport(currentWeather), which gives users an overview of stored food data and current weather-related information. These features improve the system's completeness and usability beyond basic CRUD.	FoodManager.cpp: saveFavoriteFood(), viewFavorites(), generateSummaryReport(); main.cpp: cases 3, 4, and 9

2.2 Non-Functional Requirements

Non-functional requirements describe the quality standards, technical constraints, and performance expectations of the WEASOLVE system. These requirements focus on how the system operates rather than only what the system does. For this project, the major non-functional requirements are robustness, modularity, readability, maintainability, data integrity, and resource efficiency.



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

The system was designed to remain stable in the presence of invalid user input, maintain a clean multi-file architecture, protect class data through encapsulation, and preserve food records through file handling. These requirements help ensure that WEASOLVE is not only functional, but also reliable, understandable, and suitable for academic evaluation.

NFR ID	Requirement	Rubric Definition	System Implementation and Evidence from Code	Source Files / Functions
NFR1	Robustness and Defensive Programming	The system must handle invalid inputs and unexpected user actions without crashing, entering infinite loops, or corrupting program data.	WEASOLVE uses a dedicated InputValidator module to validate user input before processing. Numeric menu input is handled through InputValidator::getIntInRange(), which prevents invalid menu selections. Weather and budget entries are limited to the accepted categories: sunny, rainy, cold, low, mid, and high. The system also allows cancel commands, such as 0 in specific input prompts. Invalid input produces clear error messages and returns the user safely to the correct flow.	InputValidator.h; InputValidator.cpp: getIntInRange(), getWeatherInputAllowCancel(), getBudgetInputAllowCancel(), getPositiveDoubleAllowCancel(), getNonEmptyLine(), isCancelCommand()
NFR2	Input Stream Recovery	The system must prevent type-mismatch errors from causing crashes or infinite input loops.	The input validation system uses stream recovery techniques such as clearing invalid cin states and flushing invalid buffer contents. This prevents common C++ console input errors, such as entering letters when the system expects a number. Instead of terminating, the program displays an error message and asks for valid input again.	InputValidator.cpp: numeric input validation functions



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

NFR3	Structural Modularity	The system must be organized into multiple source files and header files with clear responsibilities.	WEASOLVE follows a multi-file architecture. Each major component is separated into its own .h and .cpp files. main.cpp controls the program loop and user navigation. FoodItem defines the food entity. FoodManager handles record management and file operations. RecommendationEngine handles smart ranking. RecommendationRule defines the polymorphic scoring rules. InputValidator handles user input. This separation improves maintainability and readability.	main.cpp; FoodItem.h/.cpp; FoodManager.h/.cpp; RecommendationEngine.h/.cpp; RecommendationRule.h/.cpp; InputValidator.h/.cpp; ; UserPreference.h; RecommendationResult.h
NFR4	Encapsulation and Data Protection	Class attributes must be protected from direct external modification and accessed through controlled public methods.	The FoodItem class stores food details such as ID, name, weather, budget level, price, restaurant, and health note as private attributes. These values are accessed and modified through public getters and setters. This protects the internal state of each food object and follows object-oriented programming principles.	FoodItem.h; FoodItem.cpp



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

NFR5	Object-Oriented Design and Polymorphism	The system must demonstrate object-oriented programming concepts such as classes, abstraction, inheritance, and polymorphism.	WEASOLVE demonstrates OOP through multiple classes and object interactions. The RecommendationRule class acts as an abstract base class. WeatherMatchRule, BudgetMatchRule, and AffordabilityRule inherit from it and override the score() and explain() functions. The RecommendationEngine stores these derived rule objects through base-class pointers using <code>unique_ptr<RecommendationRule></code> , allowing the engine to process multiple rules polymorphically.	RecommendationRule.h; RecommendationRule.cpp; RecommendationEngine.h; RecommendationEngine.cpp
NFR6	Readability and Naming Standards	The source code must use meaningful names and a consistent naming style.	The system uses descriptive class and function names such as FoodManager, FoodItem, RecommendationEngine, InputValidator, recommendSmartRanking(), loadFromFile(), saveToFile(), and generateSummaryReport(). These names make the code easier to understand and explain during defense. Class names follow PascalCase, while function and variable names generally follow camelCase.	All source files



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

NFR7	Data Persistence and File Reliability	The system must preserve important data by storing and retrieving records from an external file.	WEASOLVE uses database.txt as its external text file database. The system loads records from the file at startup and saves records back to the file when the user selects Save and Exit. The database uses a consistent pipe-delimited format, allowing the system to parse each record reliably.	FoodManager.cpp: loadFromFile(), saveToFile(); database.txt
NFR8	Practical Recommendation Integrity	The system must avoid producing misleading or impractical recommendations.	The Smart Ranked Recommendation Engine includes an affordability hard filter. If the food price exceeds the user's available money, the item is skipped before scoring. This prevents the system from displaying food that the user cannot buy. The HealthNote field is also treated only as descriptive information and is not used as a medical scoring rule, preventing unsupported health claims.	RecommendationEngine.cpp: recommend(); RecommendationRule.cpp; FoodManager.cpp: recommendSmartRanking()
NFR9	Resource Efficiency	The system must use appropriate data structures and avoid unnecessary memory waste.	The system uses vector<FoodItem> to store food records dynamically. This is more flexible than a fixed-size array because the number of food records can grow or shrink through CRUD operations. The recommendation rules are stored using unique_ptr, which helps manage dynamically allocated rule objects safely. Local variables are declared within the functions where they are needed.	FoodManager.h; FoodManager.cpp; RecommendationEngine.h; RecommendationEngine.cpp



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

NFR10	Academic and Portfolio Readiness	The system must be organized and documented in a way that supports evaluation, presentation, and future improvement.	The project is organized into clear folders such as CODE, INPUT_DATA, and DOCUMENTATION. The README explains the system's purpose, features, architecture, scoring model, database format, contributors, and future improvements. The documentation includes flowcharts and requirement analysis to support the final defense.	GitHub repository structure; README.md; DOCUMENTATION / folder
-------	----------------------------------	--	--	--

III. DESIGN SPECIFICATION

The design specification describes how the WEASOLVE system is structured and how its major components interact. This section includes the algorithmic flowcharts, file structure, database format, and detailed explanation of the Smart Ranked Recommendation Engine.

WEASOLVE follows a modular design. The program does not place all the logic in a single source file. Instead, the system separates responsibilities across different classes and files. The main controller manages the menu and user interaction; the food manager handles record and file operations; the food item class represents each food entity; the input validator handles safe input; and the recommendation engine performs the scoring and ranking logic.

This design improves maintainability because each component has a clear responsibility. If one part of the system needs improvement, it can be modified without rewriting the entire application.



3.1 Algorithmic Flowcharts

The WEASOLVE system uses two separate algorithmic flowcharts:

Figure 1. WEASOLVE Main System Flowchart

Figure 2. Smart Ranked Recommendation Engine Flowchart

The first flowchart summarizes the system's overall menu-driven structure. It shows how the program starts, loads the database, displays the main menu, accepts user choices, and routes the user to different system modules.

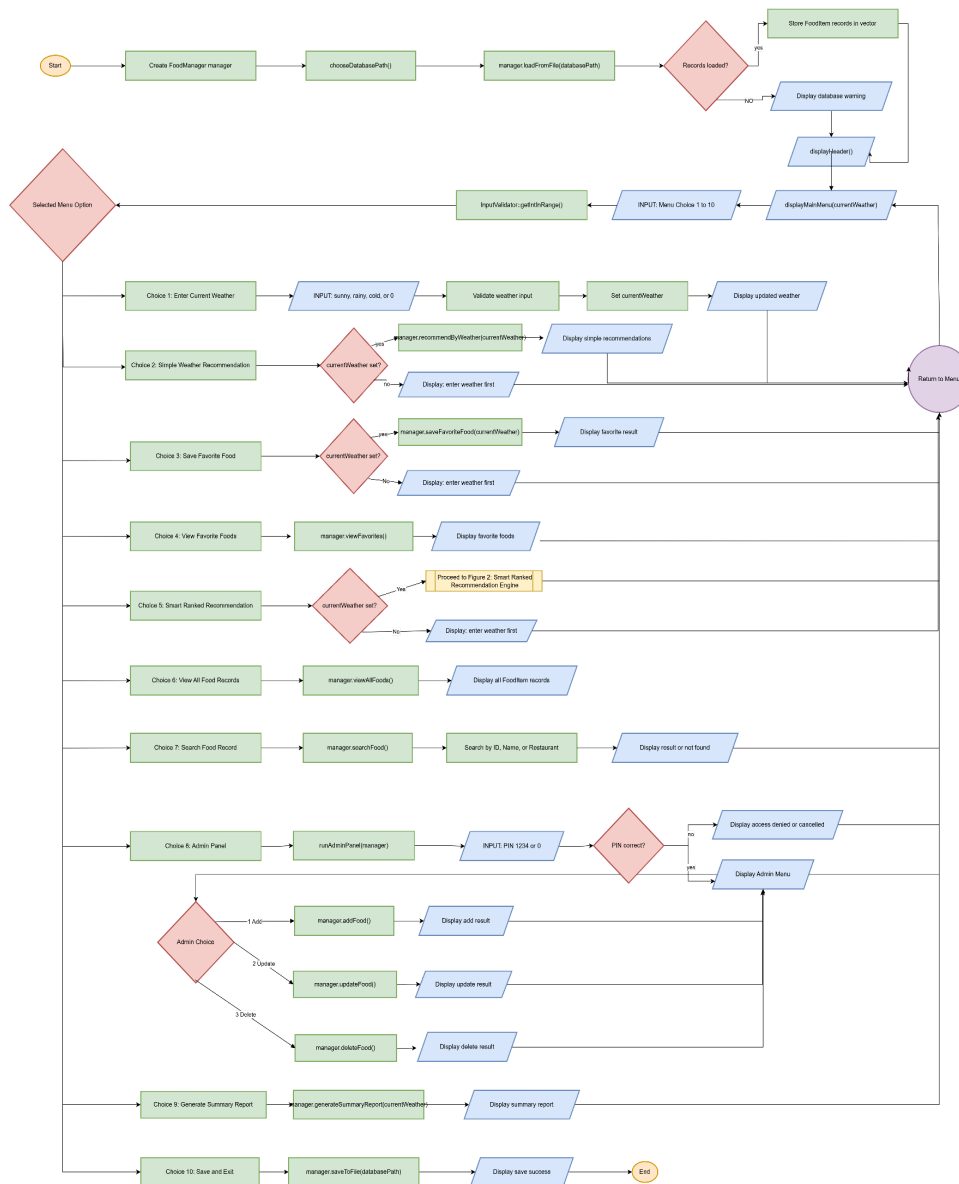
The second flowchart focuses only on the Smart Ranked Recommendation Engine. This separate flowchart is necessary because the recommendation engine is the most unique and value-added part of the system. Separating it from the main system flowchart improves readability and allows the recommendation logic to be explained in detail.



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

Figure 1 presents the main menu-driven flow of WEASOLVE.

The system creates a FoodManager object, loads records from database.txt, displays the main menu, validates the user's choice, and routes the selected option to the proper module. Option 5 directs the user to the separate Smart Ranked Recommendation Engine flowchart shown in Figure 2.

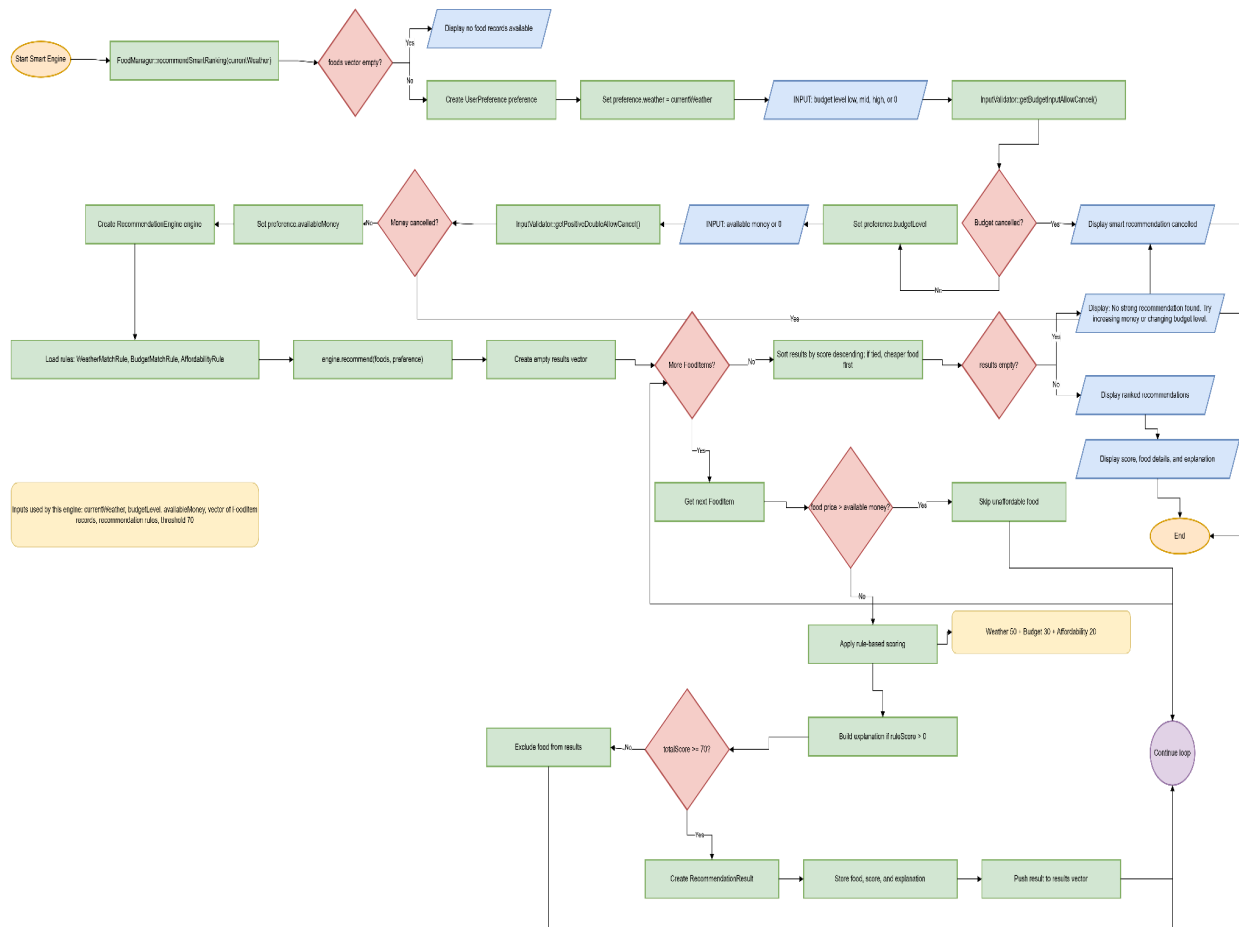




Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

Figure 2 presents the internal process of the Smart Ranked Recommendation Engine.

The system receives the current weather, budget level, available money, and food records, then evaluates each FoodItem using the rule list inside RecommendationEngine. The engine filters unaffordable items, applies the scoring rules, stores qualified RecommendationResult objects, sorts the results, and displays ranked recommendations.



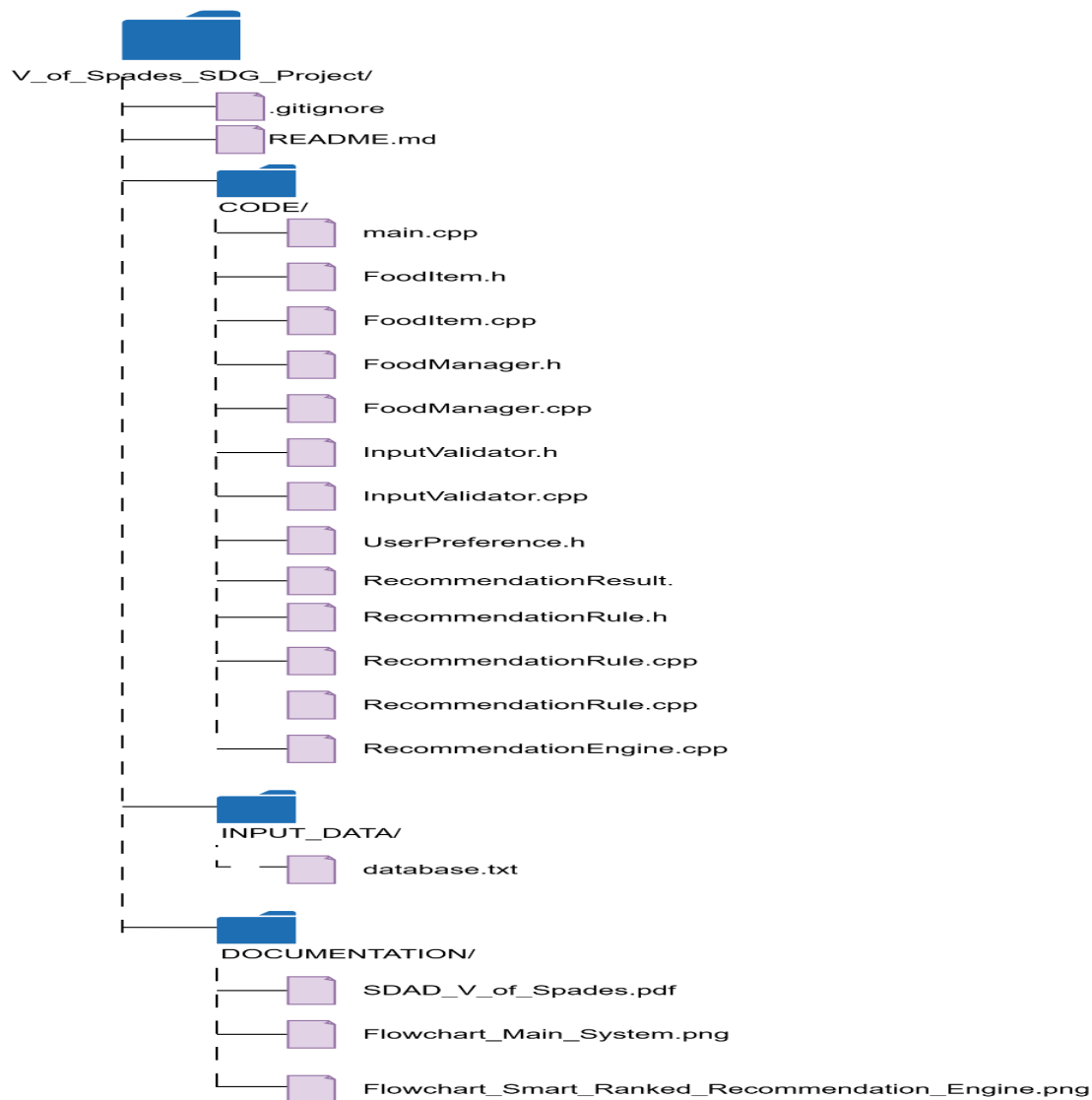
A clearer copy of the WEASOLVE flowcharts is available through the following link:

https://drive.google.com/file/d/1KOOfadYRrQIM3jcO_E3KTKg9l08iDPfp/view?usp=sharing



3.2 File Structure Diagram

The WEASOLVE project is organized into separate folders for source code, input data, and documentation. This structure improves readability and makes the project easier to evaluate in GitHub.



A clearer copy of the WEASOLVE File Structure is available through the following link:

https://drive.google.com/file/d/1opEKwbadR-DSAxvs_NX1FZvaTMpfs_P2/view?usp=sharing



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

Component Breakdown

main.cpp

Serves as the main controller of WEASOLVE by managing program startup, database loading, menu navigation, admin panel access, module routing, saving, and program exit.

FoodItem.h / FoodItem.cpp

Defines and implements the FoodItem class, which stores each food record's ID, name, weather category, budget level, price, restaurant, and health note using encapsulated attributes with getters and setters.

FoodManager.h / FoodManager.cpp

Manages the internal vector<FoodItem> collection, including file loading, file saving, CRUD operations, search, favorites, summary report generation, simple recommendations, and smart-ranked recommendations.

InputValidator.h / InputValidator.cpp

Provides reusable input validation functions for menu choices, weather input, budget input, money input, text input, and cancel commands to prevent invalid entries and type mismatch errors.

UserPreference.h

Defines a simple data container that groups the user's selected weather, budget level, and available money for use in the recommendation engine.

RecommendationResult.h

Defines the result container used by the recommendation engine to store a recommended food item, its total score, and its explanation.



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

RecommendationRule.h / RecommendationRule.cpp

Defines the abstract RecommendationRule base class and implements the derived scoring rules:

WeatherMatchRule, BudgetMatchRule, and AffordabilityRule.

RecommendationEngine.h / RecommendationEngine.cpp

Implements the Smart Ranked Recommendation Engine by applying polymorphic rules, computing recommendation scores, filtering qualified results, and sorting ranked food suggestions.

database.txt

Serves as the external pipe-delimited text file database that stores 50 food records and allows the system to load and preserve data across program sessions.

3.3 Database Format

WEASOLVE uses an external text file named database.txt as its database. Each line in the file represents one food record. The values are separated using the pipe symbol |.

The database format is:

ID|Name|Weather|Budget|Price|Restaurant|HealthNote

Example record:

1|Chicken Arroz Caldo|rainy|low|65.00|Lugawan sa Kanto|Warm rice porridge that is easy to digest and comforting during rainy weather

Each field has a specific purpose:

Field	Description
ID	Unique number used to identify the food record



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

Name	Name of the food item
Weather	Weather category where the food is recommended, such as sunny, rainy, or cold
Budget	Budget category of the food item, such as low, mid, or high
Price	Price of the food item in Philippine pesos
Restaurant	Name of the restaurant or food source
HealthNote	Informational description of the food item

The database contains 50 pre-existing food records, which exceeds the minimum requirement of 20 records. The records are loaded into the program through `FoodManager::loadFromFile()` and saved through `FoodManager::saveToFile()`.

The HealthNote field is not used as a medical rule. It only provides descriptive information about the food item. This prevents the system from making unsupported medical claims.

3.4 Smart Ranked Recommendation Engine Design

The Smart Ranked Recommendation Engine is the main value-added feature of WEASOLVE. Its flow is already shown in Figure 2, while this section explains the design reason behind the scoring model.

The engine was conceptually guided by rule-based recommendation systems, such as the Metabase rule-based recommendation reference. In a rule-based recommendation approach, items are evaluated according to predefined rules, and each rule contributes to the final recommendation. WEASOLVE adapted this concept by using three factors aligned with the system's purpose: weather, budget, and affordability.

Rule	Score	Reason
------	-------	--------



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

WeatherMatchRule	50 points	Weather is the primary basis for WEASOLVE, so it carries the highest weight.
BudgetMatchRule	30 points	Budget reflects the user's preferred spending level.
AffordabilityRule	20 points	Affordability ensures the food can realistically be purchased.
Total	100 points	Complete recommendation score.

The group chose the 50-30-20 distribution to match the project's purpose. Weather has the highest score because the system is weather-based. Budget has the second-highest score because it narrows the food recommendations based on the user's spending preferences. Affordability has 20 points because the user's actual money must still be taken into account.

Only food items with a total score of 70 or higher are included in the final recommendation list. The 70-point threshold is an implementation decision based on a 100-point relevance scale. In WEASOLVE, a score of 70 means the food item meets enough important criteria to warrant a strong recommendation.

The system also applies an affordability hard filter before ranking. If the food price exceeds the user's available funds, the item is skipped and will not appear in the final results. This prevents the system from recommending food that the user cannot actually buy.

The HealthNote field is used solely for informational purposes. It is not part of the scoring system. The group removed HealthSuitabilityRule because WEASOLVE is not a medical application and does not provide medical or nutritional prescriptions.



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

The Metabase reference supports the general concept of rule-based recommendation logic and weighted decision factors. However, the exact 50-30-20 scoring distribution and the 70-point threshold were designed by the group to fit WEASOLVE's final code implementation.

IV. TEST CASES AND VALIDATION

Testing and validation were performed to confirm that WEASOLVE works correctly, handles invalid input safely, loads and saves records properly, and produces accurate recommendation results. The tests focused on the major system requirements: database loading, input validation, CRUD operations, smart recommendation scoring, affordability filtering, and file persistence.

Test Case ID	Test Case	Input / Condition	Expected Output	Result
TC1	Database Loading Test	Run the program with database.txt available	The system loads food records into the internal vector<FoodItem> collection	Passed
TC2	Minimum Record Verification	Check the number of pre-existing records in database.txt	The database must contain at least 20 records	Passed; database contains 50 records
TC3	Main Menu Input Validation	Enter an invalid menu input, such as letters or numbers outside 1-10	The system rejects the input and asks again without crashing	Passed
TC4	Weather Input Validation	Enter an invalid weather category	The system accepts only sunny, rainy, cold, or 0 to cancel	Passed



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

TC5	Budget Input Validation	Enter an invalid budget category	The system accepts only low, mid, high, or 0 to cancel	Passed
TC6	View All Records	Select menu option 6	The system displays all stored food records	Passed
TC7	Search Food Record	Search by ID, name, or restaurant	The system displays the matching food record or a not found message	Passed
TC8	Add Food Record	Use the admin panel to add a valid food item	The new food record is added to the vector	Passed
TC9	Update Food Record	Use the admin panel to update an existing food record	The selected food record is modified correctly	Passed
TC10	Delete Food Record	Use the admin panel to delete an existing food record	The selected food record is removed from the vector	Passed
TC11	Admin PIN Validation	Enter the wrong PIN or cancel using 0	The system denies access or returns to the main menu	Passed
TC12	Simple Weather Recommendation	Enter the weather, then select the simple recommendation	The system displays food items that match the selected weather	Passed
TC13	Smart Recommendation Perfect Match	Weather: rainy, Budget: low, Money: 100	The system displays ranked food recommendations with strong scores	Passed
TC14	Affordability Hard Filter	Enter money lower than the available food prices	The system does not recommend unaffordable food	Passed
TC15	No Strong Recommendation Case	Use preferences that do not produce a score of at least 70	The system displays a no strong recommendation message	Passed



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

TC16	Save and Exit Test	Select menu option 10	The system saves records to database.txt and exits safely	Passed
------	--------------------	-----------------------	---	--------

Validation Summary

The testing results confirm that WEASOLVE successfully loads its external database, validates user input, performs CRUD operations, saves records, and generates ranked food recommendations. The Smart Ranked Recommendation Engine correctly applies the 50-30-20 scoring model and the 70-point threshold. The affordability hard filter also works as intended by preventing unaffordable food items from appearing in the final recommendation results.

V. CONCLUSION AND CONTRIBUTIONS

5.1 Project Conclusion

WEASOLVE successfully demonstrates a modular C++ console-based Weather-Based Food Recommendation System. The system satisfies the major requirements of the CC103 final project by implementing file handling, object-oriented programming, encapsulation, CRUD operations, input validation, a looped menu system, and a value-added computation engine.

Its main strength is the Smart Ranked Recommendation Engine, which ranks food items using weather, budget, and affordability rules. This allows the system to go beyond basic record storage by producing practical and explainable recommendations.

The project supports SDG 3 - Good Health and Well-Being by encouraging practical food decision-making based on the user's current situation. WEASOLVE does not provide medical advice; instead, it focuses on everyday well-being through realistic food suggestions.



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

Overall, the project demonstrates both technical programming skills and practical problem-solving. Its modular structure also allows future improvements such as a graphical user interface, SQL database integration, account management, and real-time weather API support.

5.2 Individual Contributions

Member	Role	Contribution
Paulino, Eurie V.	Project Leader and System Architect	Led the project direction, handled the main controller flow, system integration, admin panel structure, Smart Ranked Recommendation Engine, rule-based scoring architecture, affordability filter, GitHub organization, documentation leadership, and final defense preparation.
Alvarez, Jerome Oliver P.	Food Management and CRUD Developer	Assigned to FoodManager CRUD operations, including add, update, delete, search workflows, record management logic, data manipulation support, and FoodManager function testing.
Beli, Jhon Rey B.	Entity and OOP Design Developer	Assigned to the FoodItem class, object-oriented entity design, encapsulation, constructors, getters, setters, and object data integrity.
Malabanan, Justine Lyle R.	File Handling and Database Developer	Assigned to file handling, database loading, database saving, external database.txt structure, pipe-delimited database format, and persistence support.

The final contributor list reflects the active members and the final distribution of responsibilities. The responsibilities were adjusted to align with the actual final implementation, testing, documentation, and defense preparation for the project.

5.3 AI Use Disclosure

Artificial intelligence was used as a support tool during the development and documentation process of WEASOLVE. AI assistance was used for guidance, review, debugging support, documentation improvement, README preparation, code explanation, and project defense preparation.



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

Specifically, AI was used for the following purposes:

1. Reviewing the project structure and suggesting improvements for modular organization.
2. Helping identify possible bugs, input validation risks, and file handling concerns.
3. Explaining C++ concepts such as encapsulation, polymorphism, vectors, functions, and file handling.
4. Assisting in improving the academic tone and organization of the SDAD document.
5. Helping prepare the README file, flowchart explanations, and defense answers.
6. Suggesting test cases for invalid input, CRUD operations, database loading, and recommendation scenarios.
7. Providing guidance on GitHub repository organization and portfolio presentation.
8. Assisting in aligning the project with CC103 topics and final project requirements.

The group did not use AI as a replacement for the required programming work. The final source code, system design decisions, testing, debugging, file organization, and project submission remain the responsibility of the active project members. AI was used only for learning, review, debugging, guidance, and documentation support.

This disclosure is included to maintain academic honesty, transparency, and responsible AI use.



Republic of the Philippines
National Teachers College
School of Arts, Science, and Technology

VI. REFERENCES

Metabase. (n.d.). Rule-based recommendations. Metabase Community Posts.

<https://www.metabase.com/community-posts/rule-based-recommendations>

The National Teachers College. (2026). CC103 Computer Programming 2 course materials:

Decision control, loops, arrays, functions, structs, object-oriented programming, polymorphism, and file handling, The National Teachers College.

United Nations. (n.d.). Goal 3: Ensure healthy lives and promote well-being for all at all ages.

United Nations Sustainable Development Goals. <https://sdgs.un.org/goals/goal3>

W3Schools. (n.d.). C++ classes and objects. https://www.w3schools.com/cpp/cpp_classes.asp

W3Schools. (n.d.). C++ encapsulation. https://www.w3schools.com/cpp/cpp_encapsulation.asp

W3Schools. (n.d.). C++ inheritance. https://www.w3schools.com/cpp/cpp_inheritance.asp

W3Schools. (n.d.). C++ polymorphism. https://www.w3schools.com/cpp/cpp_polymorphism.asp

W3Schools. (n.d.). C++ files. https://www.w3schools.com/cpp/cpp_files.asp

W3Schools. (n.d.). C++ vectors. https://www.w3schools.com/cpp/cpp_vectors.asp

W3Schools. (n.d.). C++ user input. https://www.w3schools.com/cpp/cpp_user_input.asp